

Formal Verification: Coeffect Algebra

Zach Kelling, Lux Industries

December 2025

Abstract

We formalize the coeffect algebra that classifies what a computation needs from the world. Effects describe what a computation *does*; coeffects describe what it *needs*. The tensor product (list append) combines coeffects, with pure ($\emptyset$) as the identity.

1 Introduction

The coeffect algebra provides a formal framework for determining build reproducibility. A build is reproducible if and only if all its coeffects are reproducible (content-addressed I/O, TEE attestation). Non-reproducible coeffects (time, random, GPU, auth) contaminate the entire build.

2 Definitions

Definition 1 (Coeffect). *Access types: pure, filesystem (CA/non-CA), network (CA/non-CA), time, random, GPU, auth, TEE.*

Definition 2 (Coeffects). *A coeffect set (list of individual coeffects).*

3 Theorems and Axioms

Theorem 3 (empty_is_pure). *Empty coeffects are pure.*

Theorem 4 (pure_is_reproducible). *Pure coeffects are reproducible.*

Theorem 5 (tensor_pure_right). *Right identity: $r \otimes \emptyset = r$.*

Theorem 6 (tensor_pure_left). *Left identity: $\emptyset \otimes r = r$.*

Theorem 7 (tensor_assoc). *Associativity: $(a \otimes b) \otimes c = a \otimes (b \otimes c)$.*

Theorem 8 (tensor_pure_pure). *Pure tensor pure is pure: $\emptyset \otimes \emptyset = \emptyset$.*

Theorem 9 (tensor_reproducible). *Reproducible tensor reproducible is reproducible.*

Theorem 10 (time_not_reproducible). *Time access is not reproducible.*

Theorem 11 (random_not_reproducible). *Random access is not reproducible.*

Theorem 12 (ca_filesystem_reproducible). *Content-addressed filesystem access is reproducible.*

Theorem 13 (tee_reproducible). *TEE attestation is reproducible.*

4 Lean Source

```
/-
  Coeffect Algebra

  Effects: what a computation DOES to the world.
  Coeffects: what a computation NEEDS from the world.

  A pure build needs nothing external (coeffects = ).
  An impure build needs time, random, network, filesystem, etc.
  The tensor product ( = append) combines coeffects.

  Maps to:
  - hanzo/runtime: sandbox coeffect tracking
  - hanzo/attest: discharge proof generation
  - lux/node: build reproducibility checking

  Properties:
  - Pure is identity for tensor
  - Tensor is associative
  - Pure pure = pure
  - Time and random are non-reproducible
-/

import Mathlib.Data.Nat.Defs
import Mathlib.Data.List.Basic
import Mathlib.Tactic

namespace Build.Coeffect

/-- What a build needs from the world -/
inductive Coeffect where
  | pure -- Needs nothing
  | filesystem (path : String) -- Non-CA file access
  | filesystemCA (hash : Nat) -- Content-addressed file
  | network (host : String) -- Non-CA network
  | networkCA (hash : Nat) -- Content-addressed network
  | time -- Wall clock (non-reproducible)
  | random -- Entropy (non-reproducible)
  | gpu (device : String) -- GPU access (non-deterministic)
  | auth (provider : String) -- Credential access (ephemeral)
  | tee (attestation : Nat) -- TEE hardware attestation
  deriving DecidableEq, Repr

/-- A coeffect set (tensor = append) -/
abbrev Coeffects := List Coeffect

/-- Pure coeffect set (empty = needs nothing) -/
def Coeffects.pure : Coeffects := []

/-- Check if coeffects are pure -/
def Coeffects.isPure (r : Coeffects) : Bool := r.isEmpty

/-- Check if coeffects allow reproducible builds -/
```

```

def Coeffects.isReproducible (r : Coeffects) : Bool :=
  r.all fun c => match c with
    | .pure | .filesystemCA _ | .networkCA _ | .tee _ => true
    | .time | .random | .gpu _ | .auth _ => false
    | .filesystem _ | .network _ => false

--
-- THEOREMS
--

/-- Empty coeffects are pure -/
theorem empty_is_pure : Coeffects.isPure [] = true := rfl

/-- Pure coeffects are reproducible -/
theorem pure_is_reproducible : Coeffects.isReproducible [] = true := rfl

/-- Tensor with pure is identity (right) -/
theorem tensor_pure_right (r : Coeffects) : r ++ [] = r :=
  List.append_nil r

/-- Tensor with pure is identity (left) -/
theorem tensor_pure_left (r : Coeffects) : [] ++ r = r := rfl

/-- Tensor is associative -/
theorem tensor_assoc (a b c : Coeffects) : (a ++ b) ++ c = a ++ (b ++ c) :=
  List.append_assoc a b c

/-- Pure pure = pure -/
theorem tensor_pure_pure (a b : Coeffects)
  (ha : a.isPure) (hb : b.isPure) : (a ++ b).isPure := by
  simp [Coeffects.isPure, List.isEmpty_iff] at *
  simp [ha, hb]

/-- Reproducible reproducible = reproducible -/
theorem tensor_reproducible (a b : Coeffects)
  (ha : a.isReproducible) (hb : b.isReproducible) :
  (a ++ b).isReproducible := by
  simp [Coeffects.isReproducible, List.all_append, Bool.and_eq_true]
  exact ha, hb

/-- Time is not reproducible -/
theorem time_not_reproducible :
  Coeffects.isReproducible [.time] = false := rfl

/-- Random is not reproducible -/
theorem random_not_reproducible :
  Coeffects.isReproducible [.random] = false := rfl

/-- GPU is not reproducible -/
theorem gpu_not_reproducible (d : String) :
  Coeffects.isReproducible [.gpu d] = false := rfl

/-- Auth is not reproducible -/
theorem auth_not_reproducible (p : String) :

```

```

    Coeffects.isReproducible [.auth p] = false := rfl

/-- Content-addressed filesystem IS reproducible -/
theorem ca_filesystem_reproducible (h : Nat) :
  Coeffects.isReproducible [.filesystemCA h] = true := rfl

/-- TEE attestation IS reproducible -/
theorem tee_reproducible (a : Nat) :
  Coeffects.isReproducible [.tee a] = true := rfl

end Build.Coeffect

```

5 References

Source code: <https://github.com/luxfi/proofs/blob/main/lean/Build/Coeffect.lean>
 White papers: <https://papers.lux.network>